another pseudo-layout like we did with the adjacency matrix, but this time we need to process the nodes as well as the links.

Listing 6.5 Arc diagram code

```
function arcDiagram() {
  queue()
  .defer(d3.csv, "nodelist.csv")
  .defer(d3.csv, "edgelist.csv")
  .await(function(error, file1, file2) {
      createArcDiagram(file1, file2);
  });

function createArcDiagram(nodes,edges) {

    var nodeHash = {};
    for (x in nodes) {
      nodeHash[nodes[x].id] = nodes[x];
      nodes[x].x = parseInt(x) * 40;
    };
    for (x in edges) {
      edges[x].weight = parseInt(edges[x].weight);
      edges[x].source = nodeHash[edges[x].source];
      edges[x].target = nodeHash[edges[x].target];
    };

    linkScale = d3.scale.linear()
         .domain(d3.extent(edges, function (d) {return d.weight}))
         .range([5,10])

    var arcG = d3.select("svg").append("g").attr("id", "arcG")
         .attr("transform", "translate(50,250)");

    arcG.selectAll("path")
       .data(edges)
       .enter()
       .append("path")
       .attr("class", "arc")
       .style("stroke-width", function(d) {return d.weight * 2;})
       .style("opacity", .25)
       .attr("d", arc)

    arcG.selectAll("circle")
       .data(nodes)
       .enter()
       .append("circle")
       .attr("class", "node")
       .attr("r", 10)
       .attr("cx", function (d) {return d.x;})

    function arc(d,i) {
      var draw = d3.svg.line().interpolate("basis");
      var midX = (d.source.x + d.target.x) / 2;
      var midY = (d.source.x - d.target.x) * 2;
      return draw([[d.source.x,0],[midX,midY],[d.target.x,0]])
    };
  };
};
```

Creates a hash that associates each node JSON object with its ID value

Sets each node with an x position based on its array position

Replaces the string ID of the node with a pointer to the JSON object

Draws the links using the arc function

Draws the nodes as circles at each node's x position

Draws a basis-interpolated line from the source node to a computed middle point above them to the target node